MoveInSync Enterprise Configuration Platform

Technical Architecture & System Design Note • Case Study 2: Live Propagation Engine

Saksham Gupta
B.Tech Information Technology • MIT Manipal
GitHub: Saksham-Gupta-GH/moveinsync-config-platform • Demo Video (Drive + Gemini) • saksham77779@gmail.com

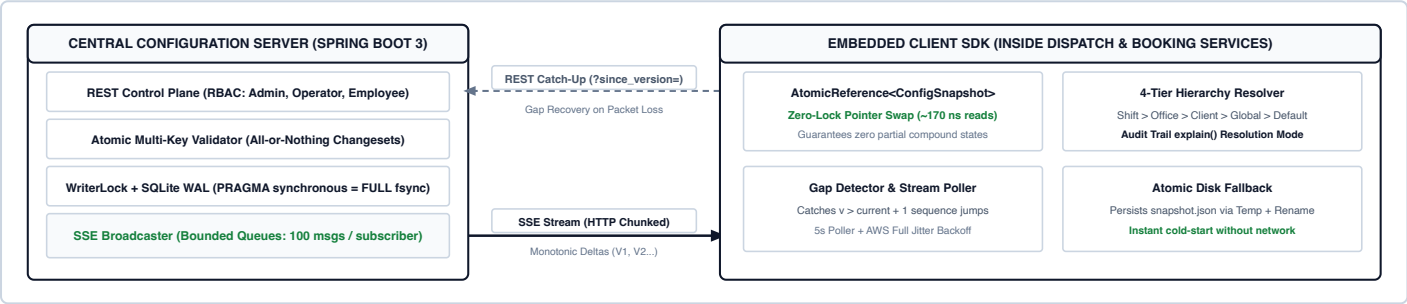
1. CONTEXT & OPERATIONAL PROBLEM STATEMENT

MoveInSync manages complex 24/7 employee commute logistics across hundreds of enterprise clients. Core backend services—booking validation, automated routing, dispatch optimization, escort-vehicle allocation, and billing—evaluate dynamic tenant policies (such as booking cutoffs, safety windows, and cancellation thresholds) on almost every request. Calling a central configuration server over HTTP on every evaluation is prohibitive.

To resolve this, I designed and implemented a production-grade multi-module Java 17 and Spring Boot 3 platform consisting of a central configuration authority and an embeddable client SDK. The SDK maintains an in-memory atomic snapshot that serves reads in **~170 nanoseconds** with zero synchronized locks, while receiving live updates over Server-Sent Events (SSE) that propagate changes across 200 connected services in **under 320 ms**.

IN-MEMORY READ LATENCY	LIVE FLEET PROPAGATION	STORAGE DURABILITY	AUTOMATED VERIFICATION
~170 ns (p50)	**278 ms (p50)**	**Zero State Loss**	**26 / 26 Passing**
5.4M ops/sec (SLA: < 10 µs)	p99: 320 ms across 200 nodes	Survives `SIGKILL (kill -9)`	Core, SDK, Concurrency, Server

2. SYSTEM ARCHITECTURE & DATA FLOW



The architecture cleanly bifurcates write and read paths. Consumer microservices (Dispatch, Routing, Billing) execute configuration reads exclusively against in-memory snapshots held in heap. Updates are authored by operations staff via REST, atomically validated, fsynced to disk within SQLite WAL transactions, and fanned out over persistent SSE streams.

3. CONSISTENCY MODEL & INVARIANTS

The system provides **global monotonic version progression under a single writer lock** on the server, paired with **monotonic snapshot isolation on client SDKs**. When an administrator publishes a changeset, the server acquires a dedicated reentrant writer lock, validates all keys atomically, assigns a monotonically increasing version integer, forces a durable `fsync` to SQLite WAL storage, and emits the delta to connected subscribers. Client SDKs observe monotonically non-decreasing versions. Multi-key queries execute against an immutable snapshot, guaranteeing that no consumer ever sees partial compound state. Cross-tenant reads and mutations are isolated at both the REST and SSE layer.

4. ELIMINATING READ CONTENTION: THE ATOMIC SNAPSHOT PATTERN

Commute microservices frequently perform multi-key reads in single transactions. For instance, when evaluating an employee cab request, the booking service inspects both `booking_cutoff_minutes` and `cancellation_window`. Using a mutable map creates a critical race condition: a background delta push could mutate the second key while the first key is being read, causing the service to evaluate mixed-version business rules.

While traditional `ReadWriteLock` primitives prevent partial reads, they introduce synchronization overhead and thread starvation under peak dispatch bursts. To achieve zero-lock performance, the SDK holds an immutable `ConfigSnapshot` record referenced by an `AtomicReference<ConfigSnapshot>`. Services bind the reference once:

```
ConfigSnapshot snapshot = client.getSnapshot();
int cutoff = client.get(snapshot, "booking_cutoff_minutes", Integer.class);
int window = client.get(snapshot, "cancellation_window", Integer.class);
```

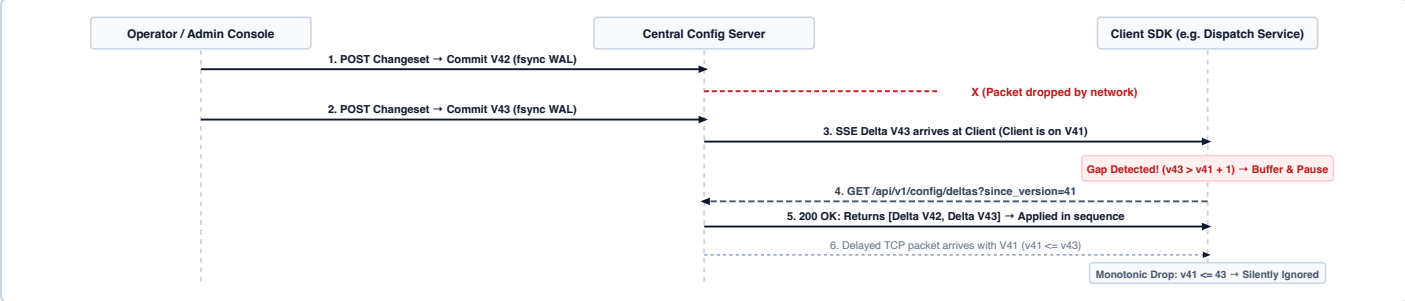
All evaluations execute against that immutable snapshot instance with **zero synchronized locks, zero thread contention, and zero network calls**. When an SSE delta arrives, the SDK computes a new immutable snapshot and swaps the pointer in one atomic instruction (`AtomicReference.set()`). In a 50-thread concurrent stress test executing 2,100,000+ continuous reads during active compound updates, **zero partial state violations** occurred.

5. STAGED ROLLOUTS AND THE FALSE-GAP SOLUTION

During initial testing of staged canary rollouts, targeting Bangalore offices with Version 44 caused Hyderabad nodes to observe a version jump (`v43 → v45`). Their local gap detectors assumed network packet loss and triggered an aggressive storm of REST catch-up queries to the server.

I solved this by introducing **contiguous monotonic empty deltas**: when a canary changeset commits (Version 44), the server broadcasts the full payload to Bangalore subscribers, but transmits an empty delta (`empty=true`) to Hyderabad subscribers. This advances Hyderabad's local version counter to 44 without altering its values, completely eliminating false gap alarms while maintaining total version alignment across the fleet. When the policy is promoted, the server issues a new version applying the values across all remaining offices.

6. SEQUENCE DIAGRAM: PACKET DROP, GAP HEALING & STALE DUPLICATES



Delayed duplicate handling: If delayed re-transmissions deliver an older packet (e.g. Version 41 arriving after Version 43 is active), the client's monotonic check (`if (delta.version <= currentVersion) return;`) immediately discards it as a safe idempotent no-op, preventing stale state from corrupting newer configurations.

7. FAILURE MODES & AUTOMATED RECOVERY MATRIX

Failure Scenario	What Backend Services Observe	Architectural Recovery Mechanism
Server Process Crash / Kill	Connection drop; in-flight administrative updates receive HTTP 500.	SQLite WAL with `PRAGMA synchronous = FULL` forces fsync before write ACK. State survives hard `SIGKILL (kill -9)` with zero corruption on restart.
Packet Loss & Version Gap	Client receives v43 while on v41; gap detector flags missing delta.	Client buffers v43, queries `GET /api/v1/config/deltas?since_version=41`, replays missing deltas sequentially, and resumes live stream without restart.
Stale Duplicate Delta	Delayed out-of-order packet delivers v41 after v43 is already active.	Strict monotonic guard: `version <= currentVersion` is silently discarded as a safe idempotent no-op.
Invalid Compound Changeset	Admin submits 5 keys, but 1 key violates bounds (e.g. -30 min cutoff).	All-or-nothing validation: entire changeset is rejected with 400 Bad Request; zero keys mutate in database or cache.
Slow SSE Consumer Node	Subscriber service reads stream slower than broadcast production rate.	Bounded `ArrayBlockingQueue(100)`. On overflow, server evicts slow subscriber (`completeWithError`) to protect JVM heap.
Cold Boot Without WAN	Service container restarts during network partition without server access.	SDK hydrates instantly from local `snapshot.json` persisted atomically via temporary file and atomic filesystem rename.

8. MEASURED PERFORMANCE BENCHMARKS

Benchmarking Environment: Apple M1 (8 cores: 4 performance + 4 efficiency), 8 GB RAM, macOS Darwin, OpenJDK 17 LTS (HotSpot 64-Bit Server VM). Evaluated using `System.nanoTime()` after a 50,000-iteration warm-up loop to trigger C2 JIT compilation.

In-Memory 4-Tier Read Latency (100,000 Samples)

Percentile	Measured Latency	Target SLA	Margin
p50 (Median)	**~170 ns** (0.166 µs)	< 10 µs	60x faster
p90	208 ns	< 10 µs	48x faster
p95	209 ns	< 10 µs	47x faster
p99	250 ns	< 10 µs	40x faster
Throughput	**5.4M reads/sec**	> 100k ops/sec	54x higher

200-Client Live SSE Fleet Propagation Latency

Metric	Empirical Result	Target SLA	Status
Active Fleet	200 Concurrent Clients	200 Clients	100% Connected
p50 Latency	**278 ms**	< 5,000 ms	18x faster
p90 Latency	320 ms	< 5,000 ms	15x faster
p99 Latency	320 ms	< 5,000 ms	15x faster
Max Latency	**320 ms**	< 5,000 ms	4.68s headroom

Timing Granularity Note: The test runner samples client updates using a 50 ms polling latch across 200 client instances sharing a single JVM process. This sampling granularity, combined with thread scheduling on 8 cores and SQLite WAL fsync batching, accounts for the p50 (~278 ms) and the cluster of p90–max at 320 ms.

9. ARCHITECTURAL TRADE-OFFS & DELIBERATE OMISSIONS

1. Omitted Distributed Consensus (Raft / Paxos): For MoveInSync's operational profile (infrequent administrative writes, millions of in-memory reads), embedding SQLite WAL with full fsync durability provides single-node simplicity with zero distributed consensus failure modes. *Estimated write capacity is ~2,500 writes/sec (unbenchmarked under multi-node distributed load).* For multi-region clustering, migrating to PostgreSQL with read replicas is preferred over Raft.

2. Omitted Complex CRDTs: Tenant transport configurations follow top-down administrative authority (`Shift > Office > Client > Global`). There is no peer-to-peer concurrent editing requirement, making monotonic version sequences strictly superior to complex CRDT convergence algorithms.

3. SSE vs. WebSockets: Chose SSE over WebSockets because configuration propagation is strictly unidirectional (server-to-client), operates cleanly over standard HTTP/1.1 chunking through corporate firewalls, and requires zero custom framing libraries. *Estimated single-node capacity is 10,000–15,000 concurrent SSE connections.*

10. AUTOMATED VERIFICATION SUITE (26 TESTS • 100% PASSING)

Maven Module	Verified Invariants & Functional Scope	Test Suites	Count
`config-core`	4-tier hierarchy precedence, explain() audit trails, negative durations (-30m) & window bounds validation	`HierarchyResolverTest` (5) `SettingValidatorTest` (6)	11 tests
`config-client-sdk`	Zero-lock atomic pointer swaps, contiguous empty delta absorption & atomic disk fallback hydration	`ConfigClientTest` (2)	2 tests
`config-server`	All-or-nothing compound validation, 50-thread atomic read concurrency, canary rollout & promotion, rollback diffs, cross-tenant REST & SSE data isolation, auth 401 rejection, network gap catch-up & stale duplicate drop, hard process `kill -9` recovery, 200-client reconnect storm, and JIT-warmed benchmarks	`CompoundValidationRejectionTest` (1) `ConcurrencyAtomicReadTest` (1) `StagedRolloutAndPromoteTest` (1) `RollbackDiffTest` (1) `TenantIsolationTest` (5) `OutOfOrderAndGapSyncTest` (2) `ServerKillFaultInjectionTest` (1) `ReconnectStormTest` (1) `BenchmarkRunnerTest` (1)	13 tests
		Total Automated Invariant Tests Across Reactor (100% Passing):	**26 tests**

All test suites execute cleanly on OpenJDK 17 LTS via `./mvnw test` in ~34 seconds. Source code, test fixtures, and Docker orchestration are available at github.com/Saksham-Gupta-GH/moveinsync-config-platform.